Week 3 Lab 2 — Predicting Heart Disease with Naive Bayes

5th November 2026

Predicting Heart Disease with Naive Bayes

The World Health Organization has estimated that 12 million deaths occur worldwide every year due to heart disease. Half the deaths in the United States and other developed countries are due to cardiovascular disease. Early prognosis of cardiovascular disease can aid decisions on lifestyle changes for high-risk patients, and in turn reduce complications. In this lab we identify the most relevant risk factors of heart disease and predict overall 10-year risk using a Naive Bayes classifier.

What is Naive Bayes?

Naive Bayes is a classification algorithm based on Bayes' theorem. It calculates the probability of each class of coronary heart disease (CHD or no CHD) given the patient's measurements, and predicts the more probable class. It is called naive because it assumes every feature contributes **independently** to the outcome. Real features are rarely fully independent, but the algorithm is fast, simple, and often works surprisingly well. We use GaussianNB, the variant for continuous measurements.

The dataset

The dataset is publicly available on Kaggle (https://www.kaggle.com/datasets/aasheesh200/framingham-heart-study-dataset) and comes from an ongoing cardiovascular study of residents of Framingham, Massachusetts. The classification goal is to predict whether a patient has a 10-year risk of future coronary heart disease (CHD). It includes over 4,000 records and 15 attributes.

Demographic

`sex_male`: male or female (nominal)

`age`: age of the patient (continuous)

`education`: education level (dropped in this lab)

Behavioural

`currentSmoker`: whether the patient is a current smoker (nominal)

`cigsPerDay`: average number of cigarettes smoked per day (continuous)

Medical (history)

`BPMeds`: on blood pressure medication (nominal)

`prevalentStroke`: previously had a stroke (nominal)

`prevalentHyp`: hypertensive (nominal)

`diabetes`: has diabetes (nominal)

Medical (current)

`totChol`: total cholesterol (continuous)

`sysBP`: systolic blood pressure (continuous)

`diaBP`: diastolic blood pressure (continuous)

`BMI`: body mass index (continuous)

`heartRate`: heart rate (continuous)

`glucose`: glucose level (continuous)

Target

`TenYearCHD`: 10-year risk of coronary heart disease ("1" = yes, "0" = no)

[1]:
```
# import the libraries we need:
# pandas      -> loading and working with the data table
# numpy       -> number handling and array operations
# matplotlib  -> plotting
# seaborn     -> the correlation heatmap and confusion-matrix heatmap
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# The variable and functions below allow the correct conversion to LaTeX tables␣
 ↪when exporting to PDF with nbconvert.
def correctness_html(value):
    return {
        'Yes': 'color: green; font-weight: bold;',
        'No': 'color: red; font-weight: bold;',
    }.get(value, '')


def correctness_latex(value):
    return {
        'Yes': 'color:{green}; bfseries:;',
        'No': 'color:{red}; bfseries:;',
    }.get(value, '')

def display_table(
    df,
    formats=None,
    latex_formats=None,
    html_style=None,
    latex_style=None,
    style_columns=None,
```

```
    hide_index=True,
    fit_to_page=False,
    na_rep=None,
):
    """
    Display a DataFrame as HTML and LaTeX for notebook viewing and PDF export.

    Formats values without modifying the DataFrame. The LaTeX version includes
    alternating row shading, bold headers, and escaped row and column labels.

    Parameters
    ----------
    df : pandas.DataFrame
        Table to display.
    formats : dict, optional
        Column names mapped to format strings or formatting functions.
        Used for both outputs unless overridden by latex_formats.
    latex_formats : dict, optional
        Formatting overrides for LaTeX output. Formatters that generate special
        characters must escape them, for example '%' as r'\\%'.
    html_style : callable, optional
        Function receiving a cell value and returning a CSS style string.
        Return an empty string to leave the cell unchanged.
    latex_style : callable, optional
        Function receiving a cell value and returning a pandas LaTeX style
        string, such as 'color:{green}; bfseries:;'.
    style_columns : list of str, optional
        Columns to which the styling functions apply. If None, apply them
        to all data cells. Callbacks receive original, unformatted values.
    hide_index : bool, default True
        Hide row indexes in both output formats.
    fit_to_page : bool, default False
        Shrink the LaTeX table if it exceeds the PDF text width.
        Does not affect the HTML display.
    na_rep : str or None, default None
        Text displayed for missing values. If None, use the default rendering
        without a custom replacement. Does not modify the underlying data.

    Returns
    -------
    None
        Displays HTML and LaTeX representations through IPython.display.

    Notes
    -----
    The PDF template must load xcolor and colortbl for LaTeX styling,
    and adjustbox when fit_to_page=True.
```

```
    Rerun and save the cell outputs before exporting without execution.
    """
    formats = formats or {}
    pdf_formats = {**formats, **(latex_formats or {})}

    html = df.style.format(formats, na_rep=na_rep, escape='html')
    latex = (
        df.style
        .format(pdf_formats, na_rep=na_rep, escape='latex')
        .format_index(escape='latex', axis=0)
        .format_index(escape='latex', axis=1)
    )

    if hide_index:
        html = html.hide(axis='index')
        latex = latex.hide(axis='index')

    # Apply LaTeX row shading and bold labels.
    def stripe_rows(values):
        return [
            'cellcolor:{gray!10}--rwrap;' if i % 2 == 0 else ''
            for i in range(len(values))
        ]

    def style_index(values):
        return [style + 'bfseries:;' for style in stripe_rows(values)]

    latex = (
        latex
        .apply(stripe_rows, axis=0)
        .apply_index(style_index, axis=0)
        .map_index(lambda value: 'bfseries:;', axis=1)
    )

    # Apply caller-supplied styles to selected columns.
    # If style_columns is None, apply to all data cells.
    if html_style is not None:
        html = html.map(html_style, subset=style_columns)

    if latex_style is not None:
        latex = latex.map(latex_style, subset=style_columns)

    latex_output = latex.to_latex()

    if fit_to_page:
        latex_output = (
            r"\begin{adjustbox}{max width=\linewidth}" "\n"
```

```
            + latex_output
            + "\n" r"\end{adjustbox}"
        )

    display({
        'text/html': html.to_html(),
        'text/latex': latex_output,
    }, raw=True)
```

The code below comes from the original lab although I have improved the plotting.

Note that, as the `education` column will not be used in the analysis, and contains `NaN` values, it would have been preferrable to first drop the column and then remove rows with 'NaN´ values afterwards.

[2]:
```
# read the Framingham heart study data - 4,238 patients
df = pd.read_csv('data/framingham.csv')

print(f"\nThe dataset contains {df.shape[0]} rows and {df.shape[1]} columns")

#print("Showing how many rows have missing values per column")
#print(df.isnull().sum())                                    # how many missing␣
 ↪values in each column?
missing_values = (
    df.isna().sum()
    .rename_axis('Column')
    .reset_index(name='Sum of missing values')
)

display_table(missing_values)

df.dropna(axis=0, inplace=True)                             # drop rows with any␣
 ↪missing value
df.drop(['education'], axis=1, inplace=True)                # keep the feature set␣
 ↪to clinical and behavioural measures
df.rename(columns={'male':'sex_male'}, inplace=True)    # clearer column name
#print(df.shape)                                             # rows and columns␣
 ↪after cleaning: (3656, 15)
print(f"\nThe dataset after the changes contains {df.shape[0]} rows and {df.
 ↪shape[1]} columns")
print("\nShowing the first rows")
df.head()                                                   # first look at the␣
 ↪cleaned data
```

```
The dataset contains 4238 rows and 16 columns
```

Column	Sum of missing values
male	0
age	0
education	105
currentSmoker	0
cigsPerDay	29
BPMeds	53
prevalentStroke	0
prevalentHyp	0
diabetes	0
totChol	50
sysBP	0
diaBP	0
BMI	19
heartRate	1
glucose	388
TenYearCHD	0

```
The dataset after the changes contains 3656 rows and 15 columns

Showing the first rows
```

[2]:
```
   sex_male  age  currentSmoker  cigsPerDay  BPMeds  prevalentStroke  \
0         1   39              0         0.0     0.0                0
1         0   46              0         0.0     0.0                0
2         1   48              1        20.0     0.0                0
3         0   61              1        30.0     0.0                0
4         0   46              1        23.0     0.0                0

   prevalentHyp  diabetes  totChol  sysBP  diaBP    BMI  heartRate  glucose  \
0             0         0    195.0  106.0   70.0  26.97       80.0     77.0
1             0         0    250.0  121.0   81.0  28.73       95.0     76.0
2             0         0    245.0  127.5   80.0  25.34       75.0     70.0
3             1         0    225.0  150.0   95.0  28.58       65.0    103.0
4             0         0    285.0  130.0   84.0  23.10       85.0     85.0

   TenYearCHD
0           0
1           0
2           0
3           1
4           0
```

[3]:
```
print("\nStatistics of the dataframe")
display(df.describe())
```

```
Statistics of the dataframe
```

```
          sex_male          age  currentSmoker   cigsPerDay       BPMeds  \
count  3656.000000  3656.000000    3656.000000  3656.000000  3656.000000
mean      0.443654    49.557440       0.489059     9.022155     0.030361
std       0.496883     8.561133       0.499949    11.918869     0.171602
min       0.000000    32.000000       0.000000     0.000000     0.000000
25%       0.000000    42.000000       0.000000     0.000000     0.000000
50%       0.000000    49.000000       0.000000     0.000000     0.000000
75%       1.000000    56.000000       1.000000    20.000000     0.000000
max       1.000000    70.000000       1.000000    70.000000     1.000000

       prevalentStroke  prevalentHyp     diabetes      totChol        sysBP  \
count      3656.000000   3656.000000  3656.000000  3656.000000  3656.000000
mean          0.005744      0.311543     0.027079   236.873085   132.368025
std           0.075581      0.463187     0.162335    44.096223    22.092444
min           0.000000      0.000000     0.000000   113.000000    83.500000
25%           0.000000      0.000000     0.000000   206.000000   117.000000
50%           0.000000      0.000000     0.000000   234.000000   128.000000
75%           0.000000      1.000000     0.000000   263.250000   144.000000
max           1.000000      1.000000     1.000000   600.000000   295.000000

             diaBP          BMI    heartRate      glucose   TenYearCHD
count  3656.000000  3656.000000  3656.000000  3656.000000  3656.000000
mean     82.912062    25.784185    75.730580    81.856127     0.152352
std      11.974825     4.065913    11.982952    23.910128     0.359411
min      48.000000    15.540000    44.000000    40.000000     0.000000
25%      75.000000    23.080000    68.000000    71.000000     0.000000
50%      82.000000    25.380000    75.000000    78.000000     0.000000
75%      90.000000    28.040000    82.000000    87.000000     0.000000
max     142.500000    56.800000   143.000000   394.000000     1.000000
```

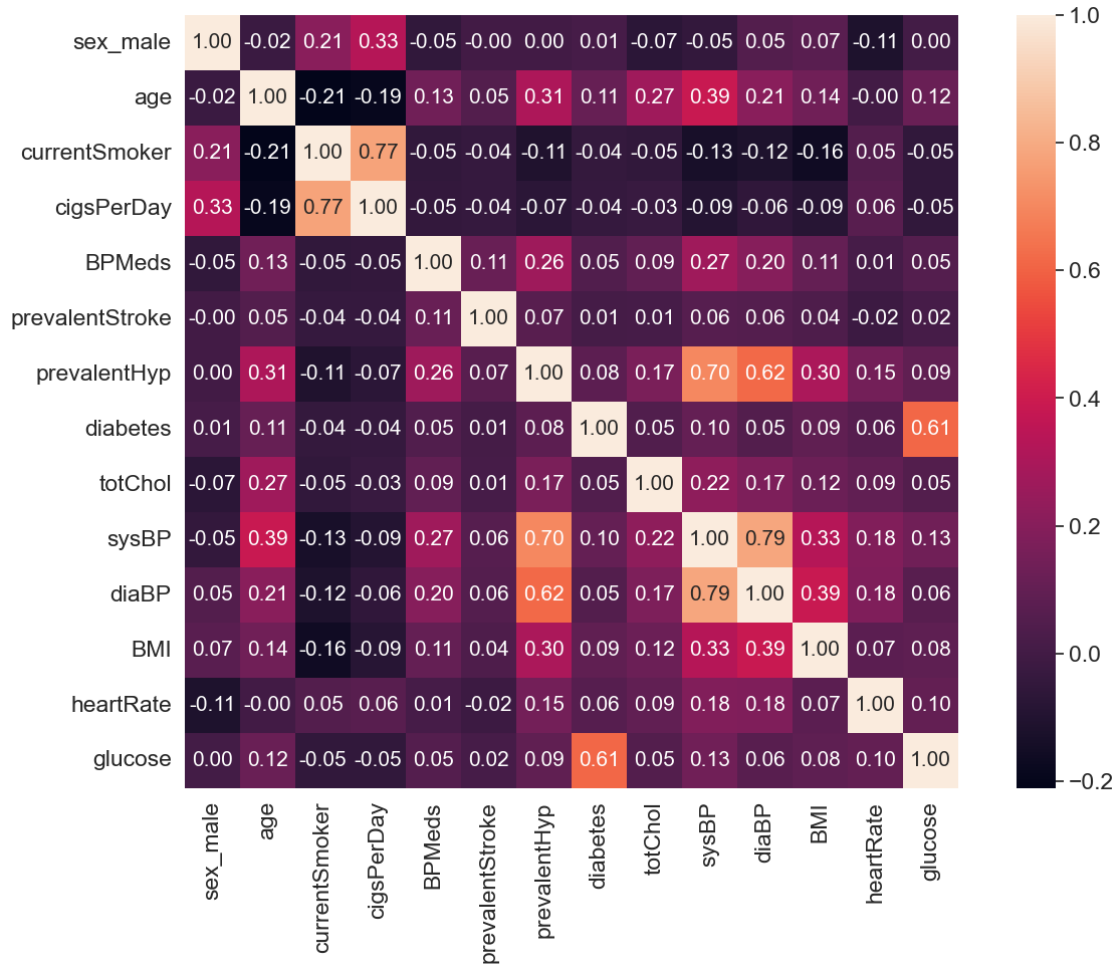
[4]:
```
# one styling call: whitegrid background, notebook sizing, larger fonts
sns.set_theme(style="whitegrid", context="notebook", font_scale=1.5)

# the 14 feature columns we will check for correlation
cols = ['sex_male','age', 'currentSmoker', 'cigsPerDay', 'BPMeds',␣
 ↪'prevalentStroke', 'prevalentHyp',
       'diabetes','totChol', 'sysBP', 'diaBP', 'BMI', 'heartRate','glucose']

# correlation between every pair of features (a 14 x 14 grid of values)
cm = np.corrcoef(df[cols].values.T)

# draw it as a heatmap: annot=True prints each value, fmt='.2f' = 2 decimal␣
 ↪places
f, ax = plt.subplots(figsize=(15,10))
heatMap = sns.heatmap(cm, cbar=True, annot=True,square=True, fmt='.2f',␣
 ↪annot_kws={'size' : 15}, yticklabels=cols,
xticklabels=cols)
```

```
plt.show()
```



Removing highly correlated features

Naive Bayes assumes every feature is independent. When two features are strongly correlated they carry almost the same information, and the model effectively counts that evidence twice. The function below scans the correlation matrix and flags any feature with a correlation above 0.7 with an earlier feature, so we can drop it.

[5]:
```
debug_mode = False
# scan the correlation matrix and collect the name of any feature whose
# correlation with an earlier feature is stronger than the threshold -
# those are the columns we will drop before fitting Naive Bayes
def correlation(dataset, threshold):
    col_corr = set()                                    # Set of all the names␣
 ↪of correlated columns
    corr_matrix = dataset.corr()
    for i in range(len(corr_matrix.columns)): # The row
```

```
        if debug_mode:
            print(f"Processing feature {i}: {corr_matrix.columns[i]}")
            if corr_matrix.columns[i] in ['cigsPerDay', 'diaBP']:
                print("Feature of interest for debugging found")
        for j in range(i): # The column
            if abs(corr_matrix.iloc[i, j]) > threshold: # we are interested in␣
↪absolute coeff value
                colname = corr_matrix.columns[i]        # getting the name of␣
↪column
                col_corr.add(colname)
                print(f"Feature '{colname}' is highly correlated with feature␣
↪'{corr_matrix.columns[j]}'")
    return col_corr

# run it on our 14 features with a 0.7 threshold and show what it found
corr_features = correlation(df[cols], 0.7)
print(f"The columns with high correlation are: {corr_features}")
```

```
Feature 'cigsPerDay' is highly correlated with feature 'currentSmoker'
Feature 'diaBP' is highly correlated with feature 'sysBP'
The columns with high correlation are: {'cigsPerDay', 'diaBP'}
```

0.0.1 Why are these features correlated?

Smoking status and cigarettes per day `currentSmoker` records **whether someone currently smokes**, while `cigsPerDay` records **how many cigarettes they smoke per day**.

These describe two aspects of the same behaviour. People who do not currently smoke generally report zero cigarettes per day, whereas current smokers generally report a positive number. This creates an expected positive association.

The relationship is therefore largely explained by how the variables are defined, rather than by one physiological process causing another.

They still convey different information: smoking status distinguishes current smokers from non-smokers, while cigarettes per day describes the intensity of smoking. Neither variable alone captures the full smoking history, such as duration or past exposure.

Systolic and diastolic blood pressure `sysBP` and `diaBP` measure arterial pressure at different stages of the same heartbeat:

- **Systolic pressure (`sysBP`):** pressure when the heart contracts and pumps blood into the arteries.
- **Diastolic pressure (`diaBP`):** pressure while the heart relaxes between beats.

Both reflect the same cardiovascular system. Shared influences—including resistance to blood flow, blood volume and the mechanical properties of arteries—help explain why they often vary together.

However, they do not always rise equally. For example, stiffening of the large arteries can increase systolic pressure more than diastolic pressure, widening the difference between them, known as **pulse pressure**.

Their correlation is therefore physiologically plausible, but they are not interchangeable measurements. The correlation alone does not establish which mechanisms explain the pattern in these particular patients.

[6]:
```
# drop cigsPerDay and diaBP - each is highly correlated with a feature we keep
# (cigsPerDay with currentSmoker, diaBP with sysBP)
df.drop(corr_features,axis=1, inplace=True)
df.head()
```

[6]:

	sex_male	age	currentSmoker	BPMeds	prevalentStroke	prevalentHyp	\
0	1	39	0	0.0	0	0	
1	0	46	0	0.0	0	0	
2	1	48	1	0.0	0	0	
3	0	61	1	0.0	0	1	
4	0	46	1	0.0	0	0	

	diabetes	totChol	sysBP	BMI	heartRate	glucose	TenYearCHD
0	0	195.0	106.0	26.97	80.0	77.0	0
1	0	250.0	121.0	28.73	95.0	76.0	0
2	0	245.0	127.5	25.34	75.0	70.0	0
3	0	225.0	150.0	28.58	65.0	103.0	1
4	0	285.0	130.0	23.10	85.0	85.0	0

[7]:
```
percentages = (
    df.iloc[:, -1]
    .value_counts(normalize=True)
    .sort_index()
    .rename_axis('TenYearCHD')
    .to_frame('Percentage')
)

display_table(
    percentages.reset_index(),
    formats={'Percentage': '{:.2%}'},
    latex_formats={
        'Percentage': lambda value: f'{value:.2%}'.replace('%', r'\%')
    },
    hide_index=True,
)
```

TenYearCHD	Percentage
0	84.76%
1	15.24%

The dataset is imbalanced: more individuals have no recorded CHD event within the 10-year period (class 0) than have an event (class 1).

This imbalance can lead to high overall accuracy even when the model identifies relatively few positive cases. We should therefore examine class 1 recall, precision, F1-score and the confusion

matrix, rather than relying on accuracy alone.

This exercise uses **stratification** to preserve approximately the same class proportions in the training and test sets. Stratification makes the splits more representative of the dataset's class distribution, but **does not balance the classes or directly improve detection of the minority class**.

If validation results show poor detection of class 1, possible approaches include threshold adjustment, oversampling or undersampling. Any resampling must be applied only to training data—and separately within each training fold when using cross-validation.

[8]:
```
# x = every column except the last (the 12 features)
# y = the last column only (TenYearCHD - the value we want to predict)
# .values converts from a DataFrame to a plain NumPy array
x = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# separate dataset into train (70%) and test (30%)
# random_state=0 makes the split reproducible - same rows every run
# stratify=y keeps the CHD rate (~15%) the same in both sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.3,␣
 ↪random_state=0, stratify=y)

# confirm the split: (rows, features) for each set
#X_train.shape, X_test.shape
split_summary = pd.DataFrame({
    'Dataset': ['Training', 'Test'],
    'Rows': [X_train.shape[0], X_test.shape[0]],
    'Features': [X_train.shape[1], X_test.shape[1]],
})

display_table(
    split_summary,
    formats={'Rows': '{:,}', 'Features': '{:d}'},
)
```

Dataset	Rows	Features
Training	2,559	12
Test	1,097	12

0.0.2 Note: feature scaling has no effect on Gaussian Naive Bayes

The original lab standardises the features with `StandardScaler`, arguing that otherwise "big-number features (e.g. `totChol` ~240) drown out small ones (e.g. `sex_male` 0/1)". **That reasoning does not apply to Gaussian Naive Bayes.**

Gaussian Naive Bayes never compares features with each other or combines them into a distance. For each feature it fits a separate normal distribution within each class (its own mean and standard deviation), then asks how likely the patient's value is under "CHD" versus "no CHD". Standardising

a feature shifts and stretches it in exactly the same way for both classes, so the stretch factor cancels out of that comparison. A cholesterol of 240 mg/dL or the same value expressed in standard deviations leads to the same prediction.

We checked this on this notebook's data and split: with and without the scaler, **all 1,097 test predictions are identical**, and the predicted probabilities differ by at most 0.00005. That tiny difference comes from `var_smoothing`, a small constant GaussianNB adds to every variance for numerical stability, which is proportional to the largest feature variance and therefore depends on scale.

The scaling step is therefore **harmless but unnecessary** here. The "large values dominate" argument is valid for models that combine features through distances or weights, such as KNN, SVMs, regularised logistic regression and neural networks. We keep the scaler only for consistency with the original lab, and because it would matter if the classifier were swapped for one of those models.

[9]:
```
use_standard_scaler_bool = True # It is unnecesary and can be set to 'False'. The original lab uses it, but the explanation given is incorrect.

if use_standard_scaler_bool:
    # standardise the features: rescale each column to mean 0, standard deviation 1,
    # so big-number features (e.g. totChol ~240) don't drown out small ones (e.g. sex_male 0/1)
    from sklearn.preprocessing import StandardScaler
    sc = StandardScaler()

    # fit_transform on TRAIN: learn each column's mean and std, then rescale
    X_train = sc.fit_transform(X_train)

    # transform only on TEST: reuse the means and stds learned from the training set
    # (never fit on the test set - that would leak test information into the model)
    X_test = sc.transform(X_test)
```

[10]:
```
# create the Gaussian Naive Bayes classifier
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()

# fit = learn from the training data. For each class (CHD / no CHD) the model learns
# how common the class is, plus each feature's mean and spread (a bell curve per feature).
# To predict, it asks: which class makes this patient's numbers most probable?
classifier.fit(X_train, y_train)
```

[10]:
```
GaussianNB()
```

[11]:
```
# ask the fitted model to predict CHD (1) or no CHD (0) for every test patient
y_pred = classifier.predict(X_test)

# show the predictions and the actual answers side by side:
# left column = predicted, right column = actual
# (reshape turns each 1-D array into a column; concatenate glues the columns␣
 ↪together)
side_by_side_matrix = np.concatenate((y_pred.reshape(len(y_pred),1), y_test.
 ↪reshape(len(y_test),1)),1)
#print(side_by_side_matrix)
print(f"The side-by-side matrix has {side_by_side_matrix.shape[0]} rows and␣
 ↪{side_by_side_matrix.shape[1]} columns.")

predicted = np.asarray(y_pred).ravel()
actual = np.asarray(y_test).ravel()

comparison = pd.DataFrame({
    'Test case': np.arange(1, len(actual) + 1),
    'Predicted CHD': predicted,
    'Actual CHD': actual,
    'Correct?': np.where(predicted == actual, 'Yes', 'No'),
})

print(f"The side-by-side comparison matrix has {comparison.shape[0]} rows and␣
 ↪{comparison.shape[1]} columns.")

display_table(
    comparison.head(20),
    html_style=correctness_html,
    latex_style=correctness_latex,
    style_columns=['Correct?'],
)
```

```
The side-by-side matrix has 1097 rows and 2 columns.
The side-by-side comparison matrix has 1097 rows and 4 columns.
```

Test case	Predicted CHD	Actual CHD	Correct?
1	0	0	Yes
2	0	0	Yes
3	0	0	Yes
4	0	0	Yes
5	0	0	Yes
6	0	1	No
7	0	0	Yes
8	0	0	Yes
9	0	0	Yes
10	0	0	Yes
11	0	0	Yes
12	0	0	Yes
13	0	0	Yes
14	0	0	Yes
15	0	1	No
16	0	0	Yes
17	1	1	Yes
18	0	0	Yes
19	0	0	Yes
20	0	0	Yes

[12]:
```
# count how the predictions line up with the actual answers
from sklearn.metrics import confusion_matrix, accuracy_score

# rows = actual, columns = predicted:
# [[ true negatives    false positives ]
#  [ false negatives   true positives  ]]
cm = confusion_matrix(y_test, y_pred)
print(cm)

# accuracy = (true negatives + true positives) / all test patients
accuracy_score(y_test, y_pred)
```

```
[[880  50]
 [137  30]]
```

[12]:
```
0.8295350957155879
```

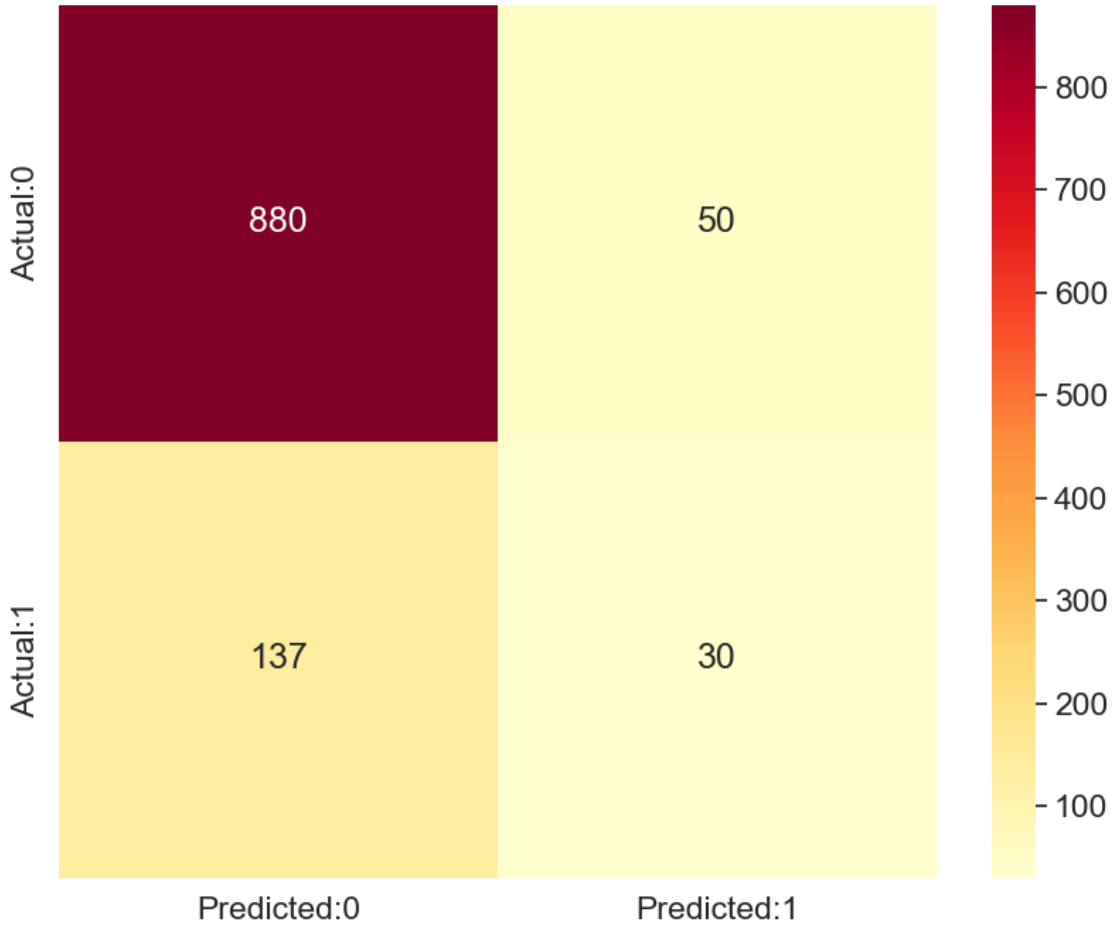
[13]:
```
# rebuild the confusion matrix (same numbers as the previous cell)
cmatrix = confusion_matrix(y_test, y_pred)

# wrap it in a DataFrame so the rows and columns carry readable labels
conf_matrix = pd.DataFrame(data=cmatrix, columns=['Predicted:0','Predicted:1'],␣
 ↪index=['Actual:0','Actual:1'])

# draw it as a heatmap: annot=True prints the count inside each square,
```

```
# fmt='d' shows the counts as whole numbers, cmap is the colour scale (yellow =␣
 ↪low, red = high)
plt.figure(figsize = (10,8))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='YlOrRd')
```

[13]: <Axes: >



Beyond accuracy

Accuracy looks respectable here — but look closer at the confusion matrix. The precision, recall and f1-score below tell the real story for the patients who actually developed CHD.

[14]:
```
# the classification report scores each class separately:
#   precision = of the patients the model flagged as this class, how many␣
 ↪really were
#   recall    = of the patients who really ARE this class, how many the model␣
 ↪caught
#   f1-score  = the balance of precision and recall (their harmonic mean)
```

```
#   support   = how many test patients are actually in this class
from sklearn.metrics import classification_report

# target_names swaps the 0/1 labels for readable ones in the printout
print(classification_report(y_test, y_pred, target_names=['No CHD (0)', 'CHD␣
 ↪(1)']))
```

```
              precision    recall  f1-score   support

  No CHD (0)       0.87      0.95      0.90       930
     CHD (1)       0.38      0.18      0.24       167

    accuracy                           0.83      1097
   macro avg       0.62      0.56      0.57      1097
weighted avg       0.79      0.83      0.80      1097
```

0.0.3 Final conclusion: confusion matrix and classification metrics

The model was evaluated on **1,097 test cases**, including **930 without a CHD event (class 0)** and **167 with a CHD event within 10 years (class 1)**.

Confusion matrix Rows represent actual classes; columns represent predicted classes.

Actual / Predicted	No CHD (0)	CHD (1)
No CHD (0)	880 true negatives	50 false positives
CHD (1)	137 false negatives	30 true positives

The model correctly classified **910 cases** and incorrectly classified **187 cases**. Most errors were **false negatives**: actual CHD cases predicted as no CHD.

Performance by class

Class	Precision	Recall	F1-score	Support
No CHD (0)	0.87	0.95	0.90	930
CHD (1)	0.38	0.18	0.24	167

For **No CHD**, the model correctly identified **880 of 930 cases**, giving a recall of approximately **94.62%**. However, among the 1,017 predictions of no CHD, **137 were actually positive cases**.

For **CHD**, the model detected only **30 of 167 cases**, giving a recall of approximately **17.96%**. It missed the remaining **82.04%**. Of the 80 positive predictions, only 30 were correct, giving a precision of **37.50%**.

The positive-class F1-score of **0.24** reflects the combination of low precision and low recall.

Overall metrics

Summary	Precision	Recall	F1-score
Macro average	0.62	0.56	0.57
Weighted average	0.79	0.83	0.80

Accuracy is 82.95%, displayed as 0.83 in the report. This is dominated by performance on the larger no-CHD class.

Macro averages give each class equal weight. Macro recall is also the balanced accuracy:

$$\text{Balanced accuracy} = \frac{880/930 + 30/167}{2} \approx 56.29\%$$

This exposes the large difference in detection rates between the two classes.

Weighted averages weight each class by its support. Because 930 of the 1,097 cases belong to class 0, these averages are influenced more strongly by performance on that class. They can therefore look reassuring despite poor CHD detection.

Comparison with an always-negative prediction Predicting no CHD for every case would achieve:

$$\text{Accuracy}_{\text{always-negative}} = \frac{930}{1097} \approx 84.78\%$$

This exceeds the model's 82.95% accuracy, but would detect **none** of the positive cases. The fitted model detects 30 positive cases at the cost of 50 false positives.

What do these results tell us? The model performs substantially better at identifying no-CHD cases than CHD cases. **A high-looking overall accuracy does not mean that it reliably identifies individuals who experience CHD within 10 years.**

If the objective is to avoid missing positive cases, **CHD recall is the key metric to improve**, while monitoring the accompanying false positives and precision.

Possible improvements should be investigated using training and validation data, including threshold adjustment, alternative models or training-only resampling. These test results should be treated as a final report rather than repeatedly used to select changes.

What did we learn?

Check before you clean: isull().sum() showed glucose alone had 388 missing values. Dropping every incomplete row cost us 582 of 4,238 records, leaving 3,656.

Naive Bayes assumes independent features: Two pairs were highly correlated — cigsPerDay with currentSmoker (0.77) and diaBP with sysBP (0.79) — so we dropped one of each pair rather than let the model count the same evidence twice.

Stratifying the split matters on imbalanced data: Only ~15% of patients developed CHD, so stratify=y keeps that rate the same in the training and test sets.

Accuracy said 83% — and it was misleading: The test set is 85% CHD-negative, so a "model" that predicts *nobody* gets heart disease scores 84.8% and beats our classifier without learning anything.

The confusion matrix and classification report told the real story: The model caught only 30 of the 167 patients who actually developed CHD (recall 0.18) and missed 137 — for a screening tool, the most dangerous kind of error.

The lesson: on imbalanced data, never judge a model by accuracy alone. Precision, recall and f1 per class show what accuracy hides.